Ajaia Docs - Project Purpose and Implementation Guide

What is this website for?

Ajaia Docs is a small shared-writing workspace. It gives a team one place to start a document, write and format a draft, import existing plain text, and let a teammate edit the same saved document.

The product goal is not to reproduce all of Google Docs. It is to demonstrate a complete, understandable slice of collaborative document work: create, edit, save, reopen, share, and recover an earlier version.

Who would use it?

- A person collecting early ideas into a working document.
- A teammate who wants to review or edit a shared draft.
- A reviewer who wants to see document creation, file import, sharing, and persistence without setting up accounts or a database.

The three built-in people are demo identities, not real registered users:

- Aria Chen (`aria@ajaia.test`)
- Morgan Lee (`morgan@ajaia.test`)
- Sam Rivera (`sam@ajaia.test`)

Use the account switcher to simulate signing in as each person. Aria owns the welcome document and has access to Morgan's sample launch brief.

Functionality you can try

Documents

- Create a blank document from the sidebar or welcome screen.
- Rename a document by editing its title.
- Open existing owned and shared documents from the sidebar.
- Filter the document list to all documents, your own documents, or documents shared with you.
- See whether a document is owned or shared, and who owns a shared document.
- Delete a document through the API. The current UI does not expose a delete button.

Writing and formatting

Write in the document canvas. The toolbar supports:

- Bold, italic, and underline.
- Paragraph, heading 1, heading 2, and heading 3.
- Bulleted lists and numbered lists.
- Block quotes.

- A live word count and a save indicator.

Edits are saved automatically after a short pause. The client captures which document was being edited when a save is queued, so quickly opening another document does not redirect that pending save to the wrong draft.

Import

Choose a `.txt` or `.md` file up to 1 MB. The app creates a new document with the file name as its title and preserves line breaks. Markdown is imported as literal text, not converted into Markdown headings or lists. Other file types and oversized files are rejected.

Sharing

1. As a document owner, open **Share**.
2. Select another demo user and invite them.
3. Switch to that user and open **Shared with me**.
4. Open and edit the shared document.
5. As the owner, remove a collaborator from the Share dialog if needed.

Owners manage access; owners and collaborators can edit. This demonstrates the sharing logic but is not real account security.

Version history

Open **History** on any document to see saved snapshots. A changed save records the previous title and content. The newest 20 snapshots are kept. Restore a snapshot to bring it back; the current draft is saved as another snapshot first, so the restore can be undone.

How the implementation fits together

```
Browser UI (HTML, CSS, JavaScript)
  |  JSON requests to /api/*
  v
Local: Express server (server.js)     Deployed: Netlify Function (netlify/functions/api.mjs)
  |              both use document-service.js
  |  validates users, inputs, document access, and sanitized HTML
  v
Local: data/documents.json            Deployed: Netlify Blobs
```

Frontend

`public/index.html` defines the workspace, navigation, editor, sharing dialog, and history dialog. `public/styles.css` provides the responsive visual design. `public/app.js` loads documents, switches demo users, imports text, queues autosaves, and drives the dialogs.

The editor uses the browser's `contenteditable` element and formatting commands rather than a third-party editor framework. This keeps installation small, with a tradeoff: it is a basic editor, not a fully featured document engine.

Backend and deployment

`document-service.js` contains the shared document operations and access rules. In local development, `server.js` serves the static frontend and Express API. On Netlify, `netlify/functions/api.mjs` handles `/api/*` requests without a long-running server. It validates the demo user, checks document access, and applies the same title, content, and HTML sanitization rules.

`netlify.toml` explicitly tells Netlify to use Node.js 24, publish the `public/` directory, and bundle the function from `netlify/functions/`. Without the publish configuration, the HTML entry point is not at Netlify's default site root

and the site may show a 404. Node.js 24 is needed by the current sanitizer/parser dependency. Netlify Blobs provides the deployed document store; the function filesystem is not used for persistence.

For Netlify, connect the repository's `piush` branch with the repository root as the base directory. Clear conflicting manual build overrides, then redeploy. The root page should serve the editor; `/api/users` should return the seeded user list. If the function reports `ERR_REQUIRE_ESM`, set Netlify's Functions environment variable `AWS_LAMBDA_JS_RUNTIME=nodejs24.x` and redeploy. Netlify deployments can be checked in the Deploy log and Functions list.

Stored document data

A document includes an identifier, title, HTML content, owner ID, collaborator list, created and updated times, last editor ID, and a bounded list of prior versions. The initial document collection includes one Aria-owned document and one Morgan-owned document shared with Aria. Local development persists that collection in `data/documents.json`; Netlify initializes and persists it in the site's `ajaia-documents` Blobs store.

Local JSON writes go to a temporary file and are renamed into place. Netlify Blobs persists data across function invocations and deploys. This is still a small demo store, not a relational database with transaction guarantees or a production concurrency strategy.

API map

- `GET /api/users` - list the demo accounts.
- `GET /api/documents?userId=...` - list documents visible to one demo identity.
- `POST /api/documents` - create a document owned by the selected identity.
- `GET /api/documents/:id?userId=...` - open an accessible document.
- `PUT /api/documents/:id` - rename and save a document; only changed content creates a snapshot.
- `DELETE /api/documents/:id` - delete a document as its owner.
- `POST /api/documents/:id/share` - grant editor access as the owner.
- `DELETE /api/documents/:id/share/:recipientId` - revoke access as the owner.
- `GET /api/documents/:id/versions?userId=...` - list history for an accessible document.
- `POST /api/documents/:id/versions/:versionId/restore` - restore a saved version for an accessible user.

The local Express API and Netlify Function both route through `document-service.js`, so validation and access rules are shared rather than reimplemented separately.

Security and scope boundaries

The app's `userId` is supplied by the browser and is only a demo selector; it is not proof of identity. Anyone who can reach the app can switch among the seeded users. Do not put private or sensitive documents in this demo, and do not treat its mock sharing as production authorization.

The server still checks the selected demo identity and document access on document routes, and strips unsupported HTML tags and attributes from saved content. The text import path escapes file text before making paragraphs, which prevents imported markup from being interpreted as executable HTML.

The local JSON file is appropriate for a small single-process demonstration. Netlify Blobs gives the deployed demo persistent storage, but this is not a substitute for a production database's query, transaction, and concurrency capabilities. Add real authentication before exposing non-demo user data.

Not implemented: real sign-in, simultaneous live co-editing, comments, suggestions, DOCX/PDF import, production database migrations, or production-grade backups.

Run and verify locally

Requirements: Node.js 20+ and npm.

```
npm install
npm test
npm start
```

Open `http://localhost:3000`. Local documents are stored under `data/`. The automated tests use temporary JSON files and a mocked Blobs store; they cover both local and serverless API behavior, including document persistence, HTML sanitization, sharing, history, and access restrictions.

Repository map

- `server.js` - Express routes, validation, access checks, sanitization, and JSON persistence.
- `document-service.js` - the API rules shared by local Express and Netlify.
- `public/` - browser UI and client behavior.
- `netlify/functions/api.mjs` - serverless `/api/*` implementation using Netlify Blobs.
- `netlify.toml` - static publish directory, function location, and bundler configuration.
- `test/server.test.js` and `test/netlify-function.test.js` - local and serverless API tests.
- `examples/launch-notes.md` - sample text file for import.
- `render.yaml` - Render deployment blueprint.
- `.github/workflows/test.yml` - GitHub Actions test workflow.
- `README.md`, `ARCHITECTURE.md`, and `AI_WORKFLOW.md` - setup and project notes.